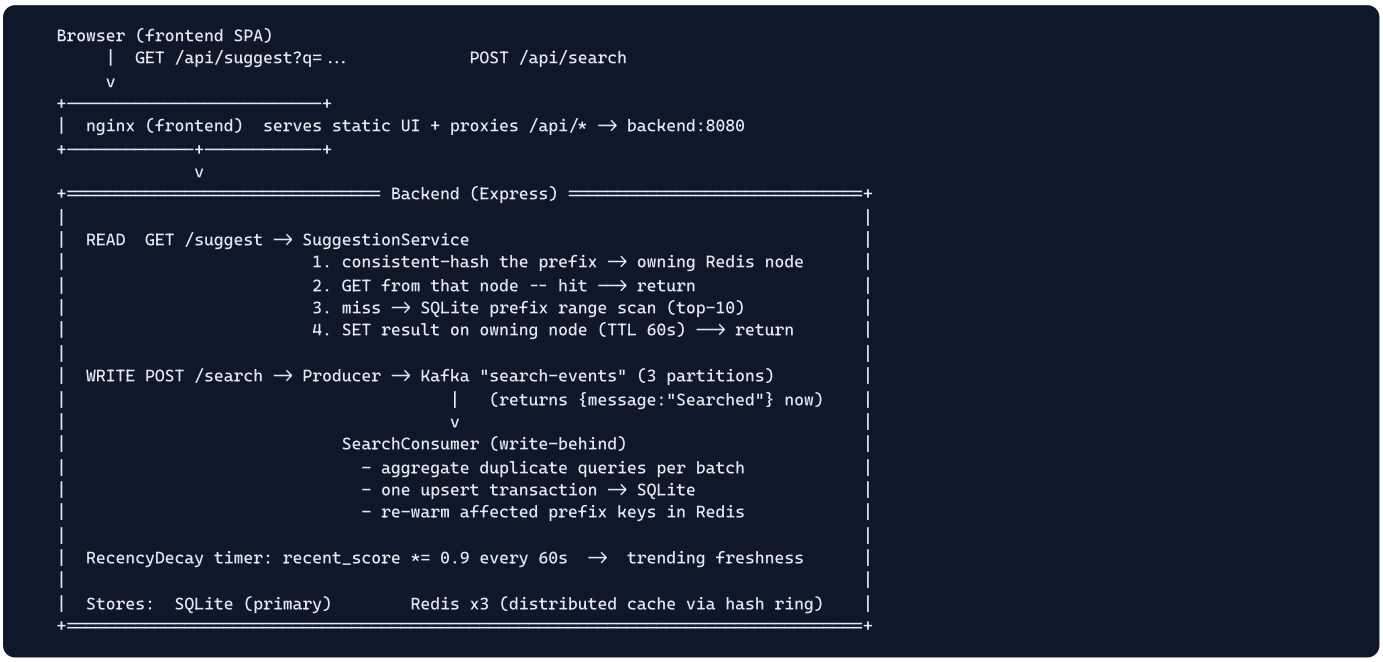
Search Typeahead — Project Report

Distributed, low-latency search autocomplete with consistent-hashed cache, write-behind batching, and recency-aware trending.

Stack: Node 20 · Express 5 · better-sqlite3 (WAL) · Redis 7 ×3 · Apache Kafka 3.8 · nginx · Docker Compose

Run: `docker compose up -d --build` → Frontend `http://localhost:3000`, API `http://localhost:8080`. The dataset is downloaded and seeded automatically on first start.

1. Architecture



Components

Component	File(s)	Responsibility
HTTP API	app.ts, routes/routes.ts	Express routes, JSON, CORS
Suggestion service	services/SuggestionService.ts	Cache-first read path, normalization
Cache cluster	cache/CacheCluster.ts, CacheNode.ts	Get/set across nodes, debug, stats
Consistent hash ring	cache/ConsistenHashRing.ts, hash.ts	Prefix → node routing
Database	db/Database.ts, seed.ts, normalize.ts	SQLite schema, queries, seeding
Write path (Kafka)	queue/KafkaClient, SearchProducer, SearchConsumer	Write-behind ingestion + batching
Batch writer (fallback)	services/BatchWrites.ts	In-memory batching when Kafka is absent
Recency decay	services/RecencyDecay.ts	Periodic score decay for trending
Metrics	metrics/Metrics.ts	Latency percentiles, hit rate, write reduction

Data model (SQLite)

```
CREATE TABLE queries (
  query        TEXT PRIMARY KEY,   -- normalized (lowercase, single-spaced)
  count        INTEGER NOT NULL,   -- all-time search count
  recent_score REAL    NOT NULL,   -- decayed recency score (drives trending)
  updated_at   INTEGER NOT NULL
);
CREATE INDEX idx_recent ON queries(recent_score DESC);
```

The `query` PRIMARY KEY is a B-tree, so prefix matching is an indexed range scan: `WHERE query ≥ @prefix AND query < @prefix||'￿'`, ordered by `count` (basic) or `recent_score` (recency), `LIMIT 10`. WAL + `synchronous = NORMAL` keep writes cheap.

Read / write / trending flows

- **Read (/suggest):** consistent hashing maps the prefix to one Redis node. Hit → instant. Miss → indexed SQLite range scan, then cache the result for 60 s.
- **Write (/search):** publishes to Kafka and returns immediately (no synchronous DB write). The consumer drains the topic in batches, collapses duplicate queries into one transaction, and re-warms affected prefixes.
- **Trending:** each search bumps `recent_score`; a timer decays all scores by `×0.9` every minute so short spikes fade. `ranking=recency` (default) sorts by this score; `ranking=basic` sorts by all-time `count`.

Distributed cache & consistent hashing

- 150 virtual nodes per physical node (450 ring points for 3 nodes); a key routes to the first ring point clockwise from `hash(key)`.
- Cache key = `sug:<ranking>:<prefix>`; value = JSON top-10; TTL = 60 s.
- Invalidation: the consumer re-warms every affected prefix key on write; the TTL bounds staleness for the long tail.
- Adding/removing a node remaps only ≈1/N of keys.

2. Dataset source & loading

Source

Peter Norvig's open-source n-gram frequency lists (from the Google Web Trillion Word Corpus): count_1w.txt (single words) and count_2w.txt (bigrams). Plain, tab-separated text.

Format & size

One record per line: `query<TAB>count` — exactly the assignment's expected input. Concatenating the word and bigram lists gives ~620,000 rows → **~591,000 unique queries** after normalization (above the 100,000 minimum).

```
the     23135851162
of      13151942776
java    55360149
iphone  50988
```

Loading — automatic (default)

The backend seeds the database on startup if it is empty. If the dataset file is missing it **downloads the dataset automatically** from the source URLs, saves it, and loads it — so `docker compose up` works on a fresh clone with no extra step (internet required; no synthetic fallback). Loading streams the file and inserts in batches of 5,000 inside SQLite transactions, so ~620k rows seed in a few seconds; duplicate counts are summed.

Loading — manual (optional)

```
node scripts/fetch-dataset.mjs              # words + bigrams → data/queries.tsv
node scripts/fetch-dataset.mjs --words-only # single words only (smaller)
```

In Docker, `data/queries.tsv` is mounted at `/seed/queries.tsv` and used if present; otherwise the auto-download writes to the data volume. **Re-seed:** `docker compose down -v && docker compose up -d`.

3. API documentation

Base URL: `http://localhost:8080` (direct) or `http://localhost:3000/api` (via the frontend proxy). All responses are JSON.

Method & path	Purpose	Response
GET /suggest?q=<prefix>&ranking=recency|basic	Top-10 suggestions	{ prefix, ranking, source, node, count, suggestions[], tookMs }
POST /search body { "query": "..." }	Record a search	{ "message": "Searched" } (400 if query missing)
GET /trending?limit=10	Trending queries	{ ranking, suggestions[] }
GET /cache/debug?prefix=<p>	Cache routing for a prefix	owner node, HIT/MISS, key hash, ring info
GET /cache/nodes	Cluster overview	nodes + ownership % + per-node stats

GET /stats	Metrics	latency p50/p95/p99, cache hit rate, write reduction
GET /health	Liveness	{ status, queries, uptimeSec }

Examples

```
curl "http://localhost:8080/suggest?q=iph&ranking=basic"
curl -X POST "http://localhost:8080/search" -H "Content-Type: application/json" -d '{"query":"iphone 15"}'
curl "http://localhost:8080/cache/debug?prefix=iph"
```

Sample responses

```
GET /suggest?q=iph
{ "prefix":"iph", "ranking":"recency", "source":"cache", "node":"redis-cache-3:6379",
  "count":10, "suggestions":[ {"query":"iphoto","count":608838,"score":608838}, ... ],
  "tookMs":1.83 }

POST /search   →  { "message": "Searched" }

GET /cache/debug?prefix=iph
{ "prefix":"iph", "ownerNode":"redis-cache-3:6379", "status":"HIT", "ttlSeconds":60,
  "consistentHashing":{ "keyHash":517625183, "virtualNodesPerNode":150, "totalRingPoints":450 },
  "valueKey":"sug:recency:iph" }
```

4. Design choices & trade-offs

Area	Choice	Why	Trade-off
Primary store	SQLite (better-sqlite3, WAL)	Zero-config, fast indexed prefix range scans, easy to demo	Single node (no replication/sharding) — fine for this scale
Cache	Read-through Redis, 60 s TTL	Read-heavy, repetitive keystrokes → sub-ms hits, shields SQLite (93.8% hit rate)	Up to 60 s stale; mitigated by re-warm on write
Cache distribution	Consistent hashing, 150 vnodes/node	Even key spread (~33/33/33); only ≈1/N remap on cluster change	Extra ring + per-node clients + a hash per lookup (negligible)
Writes	Kafka write-behind batching	/search never blocks on DB; duplicates collapse into one tx per batch (3.6× fewer writes)	Eventual consistency + small crash window before Kafka ack
Trending	recent_score + periodic ×0.9 decay	Recent activity ranks higher without permanently over-ranking short spikes	Minor background work + a second sort index
Frontend	Static SPA + nginx /api proxy	No build toolchain; same-origin (no CORS); debounced input	No component framework

Write-path failure trade-off: events buffered in Kafka survive a backend crash (durable in the log, replayed from the committed offset). The only loss window is events accepted by `/search` but not yet acknowledged to Kafka — acceptable for popularity counters where a rare lost increment is harmless. A `BatchWriter` fallback gives the same batching in-memory when Kafka is not configured (standalone local runs), trading durability for simplicity.

5. Performance report

Measured against the Docker stack via `node scripts/loadtest.mjs` + the backend's `/stats`. Workload: 3,000 `/suggest` requests over 200 prefixes (skewed, Zipf-like access), 2,000 `/search` posts over 25 distinct queries, concurrency 24. Dataset ~591,775 unique queries.

Latency (/suggest)

Metric	p50	p95	p99	avg
Server-side (3,200 samples)	**1.26 ms**	**13.33 ms**	34.37 ms	—
Client-side, warm (cache hits)	7.65 ms	13.59 ms	19.31 ms	8.26 ms
Client-side, cold (cache miss → SQLite)	44.16 ms	101.94 ms	138.35 ms	54.0 ms

Sustained throughput: **~2,897 /suggest req/s** at concurrency 24. Warm reads (Redis) are ~6× faster end-to-end than cold reads (SQLite + cache populate).

Cache hit rate

Lookups	Hits	Misses	Hit rate
3,200	3,000	200	**93.8 %**

Misses equal the number of distinct prefixes (one cold miss per key); every skewed repeat is a hit — expected for a read-through cache with a 60 s TTL under a repetitive typing workload.

Write reduction via batching

/search requests	SQLite rows written	Flushes	Reduction
2,000	555	70	**3.6×**

Each Kafka batch collapses ~6–9 duplicate events into a single transaction (e.g. log: `Kafka consumer: 9 queries → SQLite, 24 prefixes warmed`). The factor grows with arrival burstiness (faster bursts / larger batch window → more duplicates per flush).

Consistent hashing

Ring ownership (150 vnodes/node): `redis-cache-1 33.48% · redis-cache-2 33.20% · redis-cache-3 33.32%`. Routing the 200 test prefixes spread 73 / 67 / 60 across the three nodes. Adding/removing a node remaps only ≈1/N of keys.

Dimension	Result
Suggest latency (server)	p50 1.26 ms · p95 13.33 ms · p99 34.37 ms
Throughput	~2,897 req/s
Cache hit rate	93.8 %
Write reduction	3.6× (2,000 → 555 DB rows)
Ring balance	33.48 / 33.20 / 33.32 %

Absolute numbers vary with hardware and load parameters; the relationships hold: cache hits ≫ DB reads, batched writes ≪ raw writes, ring distributes evenly.